

Ring Attractor Network - Data Sheet

Overview

This document provides an overview of the **Ring Attractor Network** implemented using the **Leaky Integrate-and-Fire (LIF) model**. The network is designed to model continuous attractor dynamics with recurrent excitation and global inhibition.

1. Leaky Integrate-and-Fire (LIF) Neuron Model

File: lif_model.py

The LIF class represents a neuron modeled by the **Leaky Integrate-and-Fire** paradigm.

Key Features:

- **Membrane potential dynamics** with conductance-based synapses.
- **Synaptic interactions** (excitatory and inhibitory) with predefined time constants.
- **Noise injection** via Poisson-distributed or normal-distributed currents.
- **Spike generation** based on a voltage threshold.

Class: LIF

Attributes:

- V_{thr} ($-48mV$): Threshold potential.
- C_m ($1nF$): Membrane capacitance.
- E_l ($-70mV$): Resting potential.
- V_r ($-80mV$): Reset potential after spiking.
- τ_m ($5ms$): Membrane time constant.
- $\tau_{syn_{exc}}, \tau_{syn_{inh}}$ ($5ms$): Excitatory and inhibitory synaptic time constants.
- *synapses*: Dictionary storing excitatory (exc) and inhibitory (inh) connections.

Methods:

- `step()`: Updates the neuron's membrane potential using an Euler method, checks for threshold crossing, and handles spike generation. If the membrane potential surpasses the threshold (V_{thr}), the neuron spikes and its potential is reset to V_r . Otherwise, the potential is updated based on synaptic inputs (excitatory and inhibitory) and a stochastic noise component. The method also propagates synaptic signals to connected neurons and resets synaptic time delays after each time step.

```

def step(self,):
    # If spiked, hyperpolarize
    if self.V == 0.0:
        self.V = self.Vr

    # If above threshold, depolarize
    elif self.V >= self.Vthr:
        self.V = 0.0
        self.time_from_spike = 0

    # elif self.time_from_spike > self.tau_ref:
    #     pass

    # Else, update Current with Euler
    else:
        self.V += self.dV() * self.dt + self.noise("poisson")

    # Send time delays to connected neurons
    if self.time_from_spike > self.tau_ref:
        for neuron, weight in self.synapses["inh"].items():
            neuron.inh_ps_td.append((self.time_from_spike, weight))
        for neuron, weight in self.synapses["exc"].items():
            neuron.exc_ps_td.append((self.time_from_spike, weight))
        self.time_from_spike += self.dt

    # Reset time delays
    self.inh_ps_td.clear()
    self.exc_ps_td.clear()

```

- $dV()$: Computes voltage change due to input currents.

$$dV = \frac{-I - I_{inh} - I_{exc}}{C_m}$$

- $Il()$, $Is_{inh}()$, $Is_{exc}()$: Compute leak, inhibitory, and excitatory currents.

$$I = \frac{C_m}{\tau_m}(V - El)$$

$$I_{inh} = \sum_{n_{inh}} G_{inh}(\tau_d)(V - E_{inh}) * w_{inh}$$

$$I_{exc} = \sum_{n_{exc}} G_{exc}(\tau_d)(V - E_{exc}) * w_{exc}$$

With τ_d time from spike updated in the step function, $E_{exc} = 0 \text{ mV}$, $E_{inh} = -70 \text{ mV}$ reversal synaptic potentials, w synaptic weights

- $G_{exc}(\tau_d)$, $G_{inh}(\tau_d)$: Compute time-dependent conductance for synapses.

$$G_{inh} = \frac{\tau_d}{\tau_{inh}} e^{(1 - \frac{\tau_d}{\tau_{inh}})}$$

$$G_{exc} = \frac{\tau_d}{\tau_{exc}} e^{(1 - \frac{\tau_d}{\tau_{exc}})}$$

With τ_{inh} and τ_{exc} inhibitory and excitatory synapses time constant = 5ms

- `noise(strategy)`: Implements Poisson or normal noise models.
 - Normal : each neuron's probability to have a randomic internal activity (independent form external cues and other than leaky current) follows a gaussian distribution, centers in 0 mA and with variable sigma (ofc the noisy activity has to be on the orders of mA)
 - Poisson: each neuron receive randomic spikes independent from the external inputs or excitatory/inhibitory synapses. The distribution of these randomic spikes follow a Poissonian distribution and It's linked with τ_d : higher time-from-spike, higher the Poissonian rate of the external randomic spikes.

```
def noise(self, strategy="poisson"):
    if strategy == "normal":
        noise_current=normal(self.noise_mean, self.noise_std)
    if strategy == "poisson":
        poisson_spikes=poisson(self.noise_std*self.time_from_spike)
        noise_current=-poisson_spikes*self.kexc*(self.V - self.Eexc)* 1e-6
    return noise_current
```

2. Ring Attractor Network

File: `ring_attractor.py`

The `RingAttractor` class models a **Touretzky-style ring attractor**, which consists of recurrently connected LIF neurons.

Class: `RingAttractor`

Attributes:

- `n`: Number of neurons in the ring (121, 120 + 1 global inhibitory neuron).
- `weights`: Tuple of synaptic strengths (excitatory and inhibitory).
 - w_{ee} : excitatory connections from each exc. neuron to all the others (recursive connections of neurons in the ring)
 - w_{ei} : excitatory connections from neuron on the ring to the global inhibitory neuron
 - w_{ii} : the global inhibitory neuron auto-inhibit itself
 - w_{ie} : inhibitory connections from from the global inhibitory neuron to all the neurons in the ring

For my simulation right now I am using $w = (w_{ee}, w_{ei}, w_{ie}, w_{ii}) = (0.046, 0.046, 0.42, 0.56)$

- σ : Determines spatial connectivity profile to model excitatory connection among the ring. In my simulation, $\sigma = 0.4$
- `neurons`: List of LIF neuron objects.
- `time`: Duration of simulation.

- `raw_data, spikes`: Store membrane potential and spike activity.

Methods:

- **Touretzky_ring_connection()**: Establishes the synaptic connectivity in the ring attractor model. Each neuron (except for a designated global inhibitory neuron) forms **excitatory** synapses with nearby neurons based on a Gaussian-weighted function of distance. If the connection strength is above a threshold, the synapse is formed. Additionally, a single **global inhibitory neuron** receives excitation from all neurons and inhibits all neurons uniformly, ensuring stability and preventing runaway excitation. This function models the connectivity principles used in continuous attractor networks. I'll call from now on the synaptic weights between neuron as *syn*.

$$syn_{ee} = w_{ee} * e^{(-\frac{dist^2}{\sigma^2})}$$

$$syn_{ei}, syn_{ii}, syn_{ie} = w_{ei}, w_{ii}, w_{ie}$$

With *dist* calculated as the shortest circular distance between two neurons, computed in `ring_distances`

- `ring_distance(i, j, N)`: Computes shortest circular distance between two neurons. N: number of neuron, i and j “number” of the two neurons (numbers encode “position” on the ring, so neuron 0 is next to neuron 1 and to neuron 120). Basically, is distances with boundary conditions.

```
def ring_distance(self, i, j, N):
    return min(abs(i - j), N - abs(i - j))
```

- `simulate(process_input)`: Runs the network simulation, processes inputs, and records activity. At each `time_step`, I update each neuron potentials calling the `step()` function defined in the LIF class.

```
def simulate(self, process_input):
    if self.flushed == False:
        warnings.warn("Simulation has not been flushed!")

    self.potentials = [[] for _ in range(self.n)]
    for t in range(self.time):
        for neuron in self.neurons:
            if process_input:
                if neuron.id != (self.n-1):
                    neuron.V += self.cue[neuron.id]

            neuron.step()
            self.potentials[neuron.id].append(neuron.V)
        self.process_potentials(self.potentials)

    activity = np.any(self.spikes.iloc[:, self.time-200:self.time].sum(axis=1) != 0)
    if activity:
        bump = np.any(self.spikes.iloc[1:, self.time-200:self.time].sum(axis=1) == 0)
        if bump:
            p_opt = self.fit_result()
        else:
            # p_opt = [nan, nan, nan]
            p_opt = nan
    else:
        # p_opt = [nan, nan, nan]
        p_opt = nan

    return self.raw_data, self.spikes
```

- `process_potentials(potentials)`: Converts raw potentials into spike data.

```
def process_potentials(self, potentials):
    self.raw_data = pd.DataFrame(potentials)
    self.raw_data.index = [self.neurons[i].angle for i in self.raw_data.index]
    self.raw_data.index = self.raw_data.index.astype(int)

    self.spikes = self.raw_data == 0.0
    self.spikes = self.spikes.astype(int)
    self.spikes = self.spikes.apply(lambda x: x * x.index)
    self.spikes = self.spikes.replace(0, np.nan) #replace nan with 0

    # self.raw_data = data.copy()
    # self.spikes = spikes.copy()
    self.spikes = self.spikes.fillna(0)
    self.spikes = self.spikes.where(self.spikes == 0, 1)
    self.flushed = False
```

- `flush()`: Resets network state for a new simulation.
- `fit_result()`: Determines the mean angle of network activity (bump position).

Network Dynamics:

1. **Recurrent Excitation:** Each neuron excites nearby neurons, forming a **localized bump** of activity.
2. **Global Inhibition:** A single inhibitory unit provides uniform inhibition to all neurons.
3. **Noise-driven Dynamics:** Stochasticity is introduced via Poisson noise.
4. **Cue Inputs:** External inputs can influence network activity.

3. Summary

The **Ring Attractor Network** is a computational model of continuous attractor dynamics. It uses **LIF neurons** with structured recurrent connections to generate a stable bump of activity. This model is relevant for studies in **neuroscience**, **spatial memory**, and **robotic navigation**.

For further modifications or clarifications, refer to the class implementations in `lif_model.py` and `ring_attractor.py`.